Inception Loop

Research Project

Silas Kalinowski

Goethe University Frankfurt
Faculty of Computer Science
August 11, 2025

Contents

Introduction

Understanding neural activity in response to visual stimuli is a cornerstone of computational neuroscience. This project investigates the relationship between natural visual scenes and neural responses in ferrets by leveraging deep neural networks. Specifically, it explores the potential of these networks to identify stimuli that optimally activate neurons.

Building on the experimental framework proposed by Walker et al. (2019), this project consists of three key steps: (1) using previously recorded neural responses to natural images, (2) training neural network models on this data to predict neural responses, (3) optimizing a random input toward the most exciting input that evokes maximal neural activity in the models.

This report documents methods tested and findings obtained. It describes preprocessing techniques, evaluates different approaches, explores techniques to improve generalization and compares different neural network architectures.

The first part of the report focuses on models trained on data obtained by Nathaniel Powell in Gordon Smith's lab at the University of Minnesota, while the second part focuses on models trained on data generated by a model from Antolík et al. (2024)

Methodology and Results

2.1 Experimental Setup

The experiments were conducted using a pipeline consisting of three basic steps (see 2.1):
Firstly, natural images were presented to ferrets in an experimental setting and the neural activity of the ferrets was recorded.
Secondly, the recorded neural activity and natural images were used to train an artificial neural network to predict the neural activity of the ferrets.
Thirdly, with the objective to find the stimuli that optimally drives neurons, an initially random input image was optimized to maximize the predicted neural activation of the trained network.
The fourth step in Walker et al. (2019) pipeline (see 5.1), of presenting the most exciting stimuli to the ferret again and recording its neural activity, was not completed, since the results did not seem promising enough.

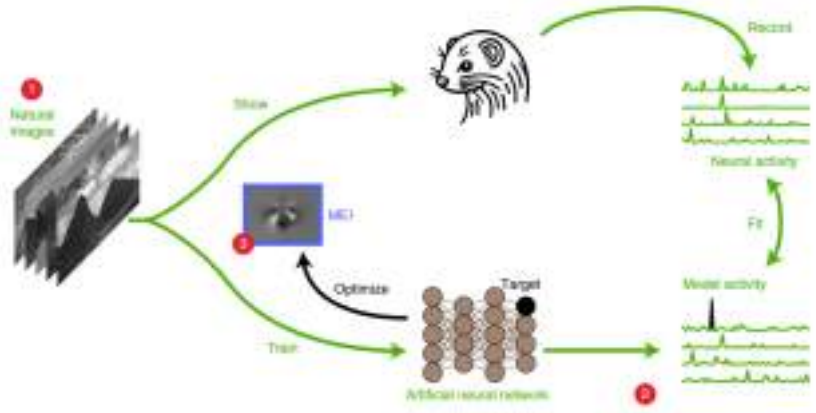


Figure 2.1: Schematic of the pipeline used

For more details about data, experimental setup and implementation, refer to section 2. The code for all experiments can be found here https://github.com/SilasKal/inceptionLoopProject.

2.2 Data

2.2.1 Data Aquisition

The data used in the first part was acquired by widefield epi-fluorescence microscopy in ferrets (n = 40, postnatal day (PD) 20 to 50). GCaMP6s (with hSyn promoter) was injected into layer 2/3 of the left brain hemisphere approximately 10 to 15 days before calcium imaging. The imaging experiments analyzed in this study were done with 15 Hz sampling rate. Analyzed images have a field of view of 320 x 270 pixels. During the imaging process the head-fixed animals have been anesthetized with isoflurane (induction with 4-5%, maintenance 1-1.5%). Heart rate, temperature and end-tidal CO2 levels have been monitored throughout the imaging sessions. Experiments were carefully performed by Nathaniel Powell in Gordon Smith's lab at University of Minnesota. A set of 150 natural (complex) visual scenes (Olmos and Kingdom 2004) have been used for V1. Stimuli have been displayed on an LCD monitor placed approx. 20 cm in front of the animal's eyes.

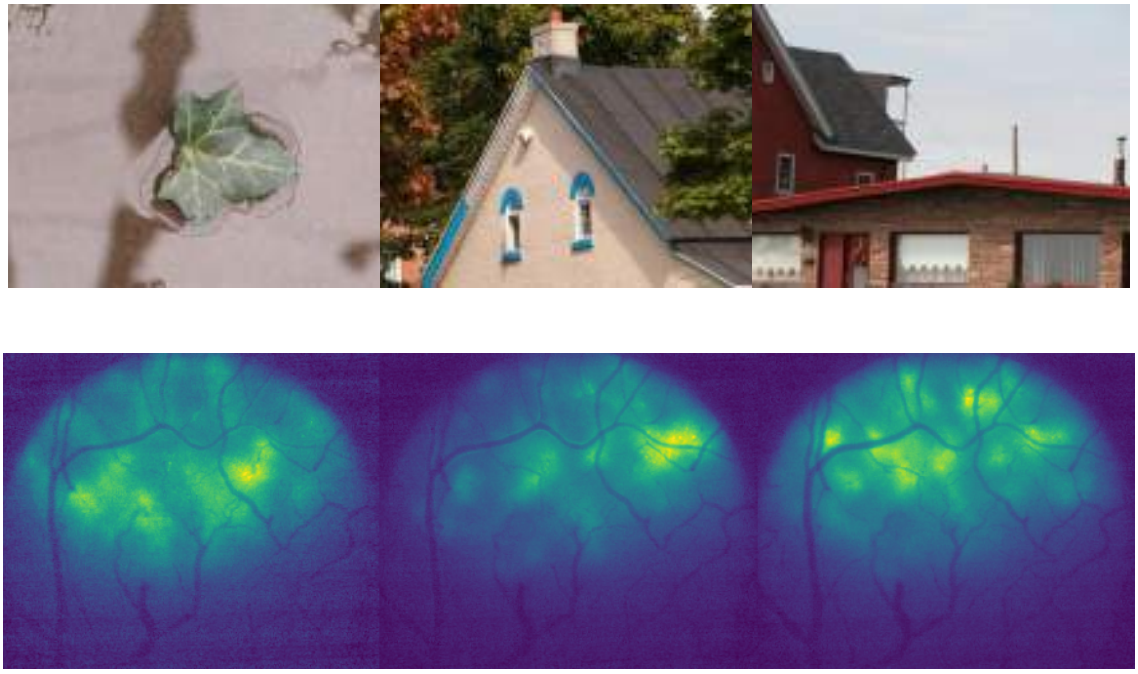


Figure 2.2: Three example samples from the data used, first row three natural images and second row three neural responses before preprocessing

2.2.2 Data Preprocessing

For each cortical area, a region of interest (ROI) was drawn manually, excluding blood-vessel artifacts. For this work, animal F0255 was selected, because it was presented with all natural images for four trials and showed high inter-trial correlation in its neural responses. F0255 was at postnatal day 40. Three images were excluded, consequently the total data consisted of four trials, each consisting of 147 pairs of natural images and recorded neural activity. Preprocessing included applying a high pass and low pass of Gaussian filters to reduce noise and only using the region of interest for the next steps [1].
For training either one of the trials or all trials were used, for more details refer to the sections below.
To assess the performance of the model on all trials reliably, k-fold cross-validation (Stone 2018) was used. In this case $k = 10$ was used. The data was therefore first split into 10 equal subsets, then, one of these subsets was selected, and the model was

[1] Code used from (Smith et al. 2018)

trained on all 9 subsets and tested on the selected subset. This was then repeated 10 times so that each subset was used for testing exactly once. It was also ensured, that no data in the test data contained any images that were also part of the training data.

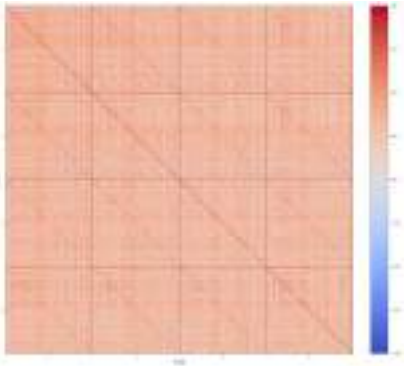


Figure 2.3: Trial Correlation for the selected animal

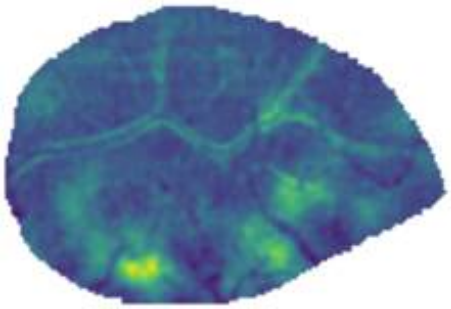


Figure 2.4: Example responses after preprocessing

2.3 Model Architectures

The **CNN** used, consisted of three convolutional layers with channel sizes of 16, 32, and 64, each using a kernel size of 3 with padding to preserve sequence length. Each convolutional layer was followed by batch normalization, ReLU activation, and a dropout layer to stabilize and regularize learning. After flattening, the model included two fully connected layers: the first reduces the feature dimension to 128, and the second maps to the final output size.

The **Transformer** model used, consisted of a stack of Transformer encoder layers applied to input data, using multi-head self-attention. After passing through the encoder stack, the model computed the mean across the sequence dimension to aggregate information, followed by a fully connected layer that projected to the final output size for prediction.

The **Simple Neural Network** used, was a fully connected neural network with three linear layers. It mapped the input to 64 units, then to 32 units, each followed by ReLU activation to introduce non-linearity. A final linear layer projected to the desired output size for prediction.

The **Deeper Neural Network** used, was a fully connected neural network with seven linear layers, progressively reducing dimensions, and finally to the output size. Each hidden layer uses ReLU activations, with dropout applied after each of the first six layers to prevent overfitting.

2.4 Model Pipelines and Evaluation

2.4.1 PCA Pipeline

The first approach used PCA to compress both the natural images and the neural activity in order to limit the computational resources needed and reduce noise with the goal of improving the efficiency and accuracy of the trained models.
Based on the percentage of variance retained with a certain number of principal components, different numbers of principal components were tested and the quality of the images and responses reconstructed from the principal components were evaluated.
For the images with 125 principal components, 98% of the variance was retained.

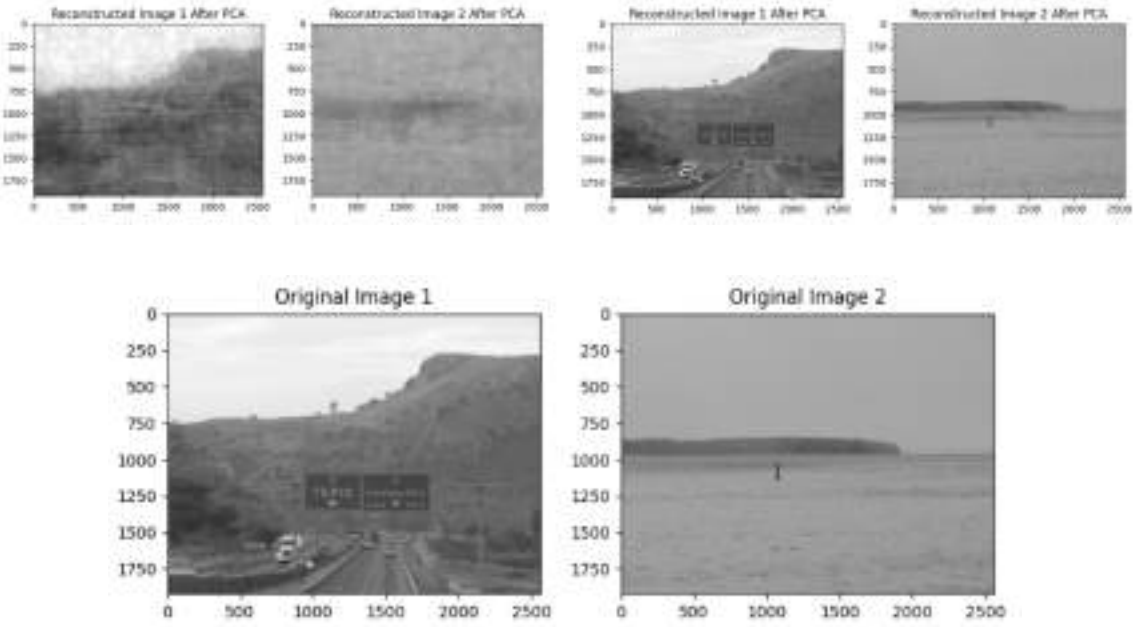


Figure 2.5: Reconstructed images from principal components: top-left from 125 PCs, top-right from 145 PCs, and bottom are the original images

The recorded neural activity was also projected onto its principal components. 50 principal components explained 87% of the variance and more than 146 principal components explained 100 percent of the variance.
Several different numbers of principal components were tested. The results below show the performance of models trained with components that retained 100% of the original variance.

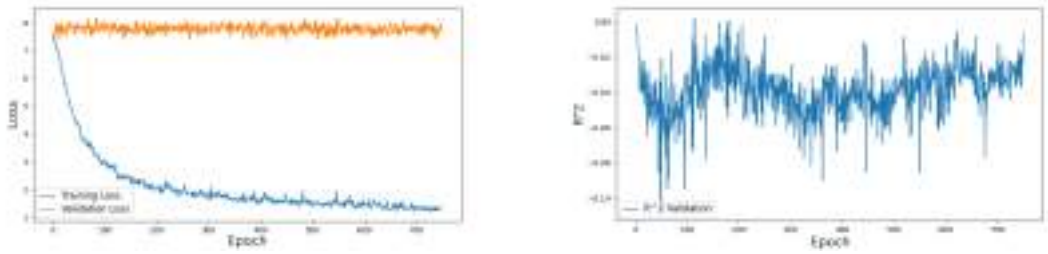


Figure 2.6: Training and validation metrics of the model trained on a single trial with principal components retaining 100 % of variance in both images and responses

As shown above, the training loss of the model trained on a single trial of images and neural activity, which had been compressed using PCA, decreases across epochs. This shows that the model is effectively learning from the training data. However, the validation loss plateaus at high value and shows no significant improvement over the training process. This and the fact that the R^2 values remain close to zero also showing no improvement indicates overfitting.
To address this issue, the entire dataset compromising of four trials was used to train the models. In total, the data now consisted of 588 images and 588 neural responses.

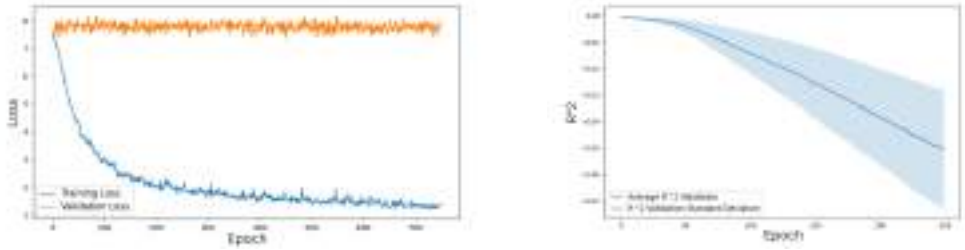


Figure 2.7: Training and validation metrics of the model trained on all four trials with principal components retaining 100 % of variance in both images and responses

Despite using the entire dataset during the training process, the model showed notable signs of overfitting and suboptimal performance metrics on the validation set, as illustrated in the plots above.

2.4.2 Pixel Pipeline

As previously explained, the PCA pipeline showed poor results. In a new approach, only relevant pixels from the neural activity data were selected. Subsequently, the model was trained to predict only these pixels based on the natural images rather than the entire neural activity. This approach reduced the complexity of the problem with the goal to improve overfitting.
To select the most relevant pixels, the following approaches were used:
The initial approach consisted of selecting pixels with the highest variance within a given trial. For initial testing, pixels with the highest variance in each of the four trials were selected, these pixels were then filtered based on their proximity or commonality across all four trials.

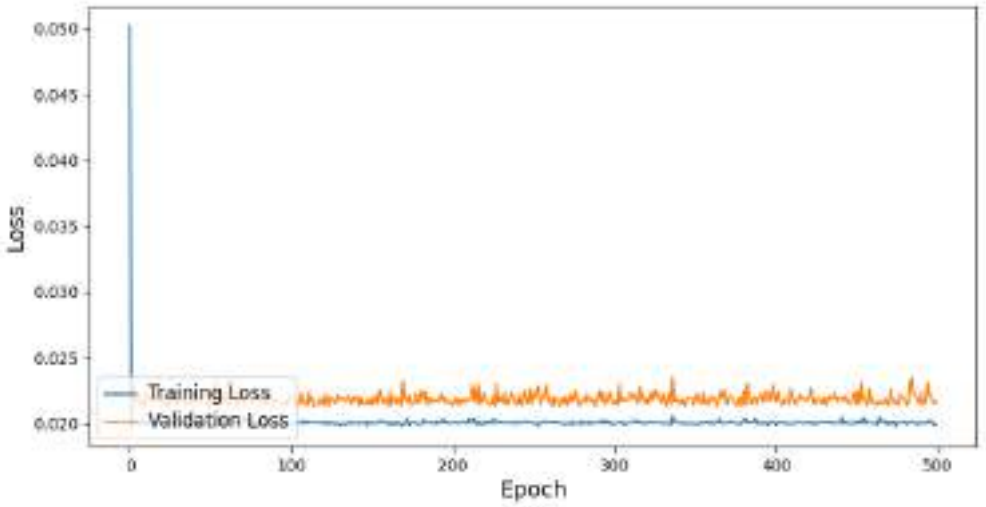


Figure 2.8: Training and validation metrics of the model trained of pixels with the highest variance from all four trials

However, the model did not demonstrate any capacity to predict this task, and the training loss remained constant throughout the entire training process. One potential explanation could be that the variance in the values of the pixels was too high over the different trials. To address this possibility, the model was retrained on a single trial.

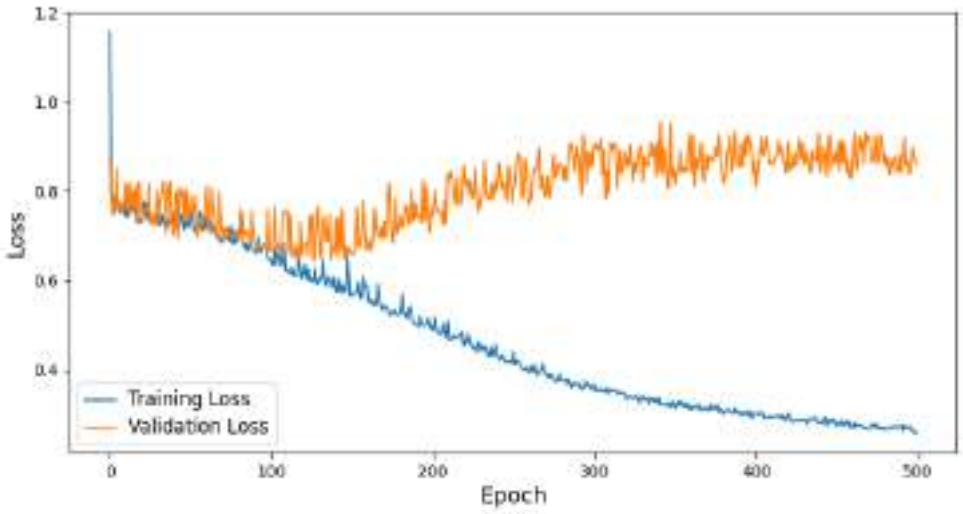


Figure 2.9: Training and validation metrics of the model trained of pixels with the highest variance from one trial

Although this did not resolve the issue initially, a deeper model trained on a single trial showed improvements in the training loss. However, the model could not generalize to the validation data.
Consequently, a new approach was investigated:
Selecting pixels with the least variance between trials. This approach was based on the assumptions that neurons with a low variance in activity across trials are the most relevant.

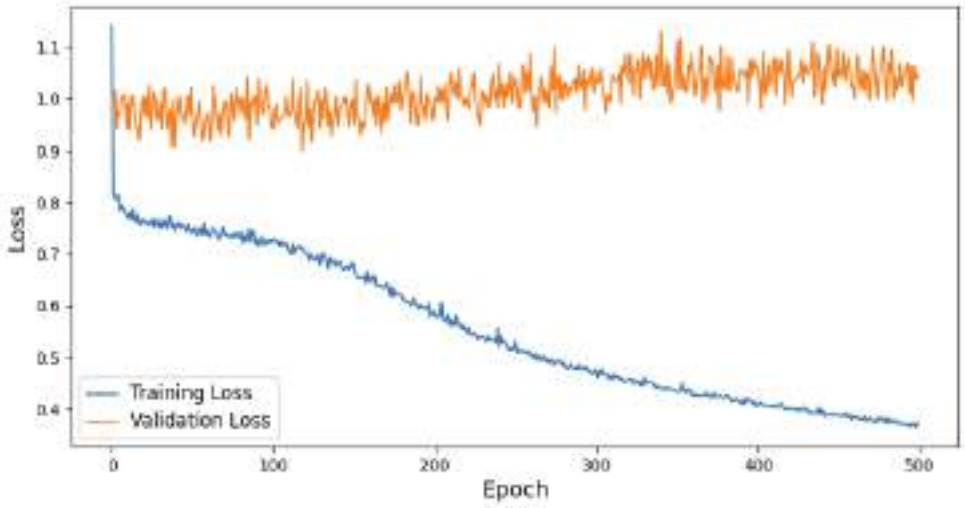


Figure 2.10: Training and validation metrics of the model trained of pixels with the lowest variance across all four trials

Unfortunately, this new approached showed similarly poor results.
The final approach consisted of combining the first and second approach with the objective again being to mitigate the overfitting of the model. This was achieved by first selecting a set of pixels with the highest variance within a trial and then only selecting the pixels with the lowest variance between trials of these pixels.

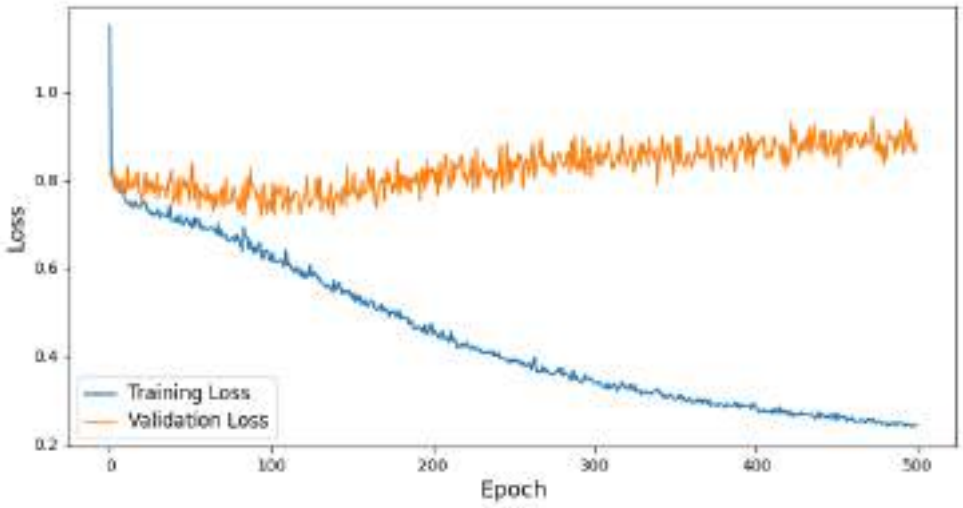


Figure 2.11: Training and validation metrics of the model trained on pixels selected with the combined approach

As shown in the training and validation metrics, this new approach did not improve the model's performance.

2.4.3 Regression with different model types

Since the neural networks showed massive signs of overfitting, smaller models were trained and tested on the same data.

Data from	Model	Training R^2	Test R^2
One Trial	Linear Regression	1.0000	-0.8321
	Ridge Regression	1.0000	-0.8320
	Support Vector Regression (SVR)	0.1786	-0.0997
	Random Forest Regressor	0.8591	-0.1130
	Gradient Boosting Regressor	0.9966	-0.2970
All Trials (k-fold CV)	Linear Regression	0.4503	Extremely Low
	Ridge Regression	0.4645	-0.2527
	Support Vector Regression (SVR)	0.1837	-0.0435
	Random Forest Regressor	0.4629	-0.0499
	Gradient Boosting Regressor	0.4634	-0.1109

Table 2.1: Training and testing R^2 values for smaller models. Metrics are shown for models trained on one trial and on all trials using k-fold cross-validation

Unfortunately again the smaller models showed either signs of overfitting or did not show any signs of learning in the training process.

2.4.4 Whole Images

Given the suboptimal outcomes of the previous approaches, an additional approach was investigated, where the uncompressed images and neural activity data were used of training.

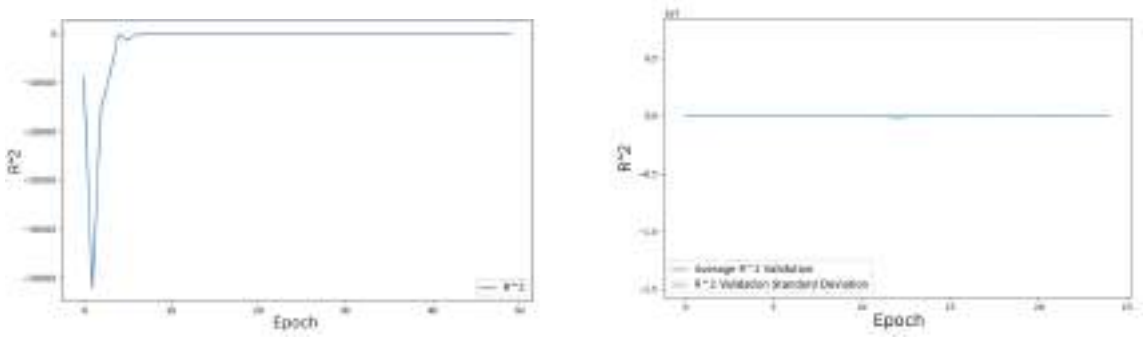


Figure 2.12: Validation R^2 Left: for a model trained on a single trial and Right: for a model trained on all four trials

As shown on the first plot, the model trained on the data of a single trial again

showed poor performance on the validation data, consequently the entire data from all four trials was used.
As seen on the second plot, training on all four trials did not rectify the overfitting problem.

2.4.5 Generalization Techniques

Data Augmentation

After the evaluation of the three main approaches, different data augmentation methods were used in an attempt to improve the poor results of each of the approaches:
Firstly, linear mixup (Zhang et al. 2018) was used. Linear mixup generates new training samples by linearly interpolating between pairs of training samples. This can improve generalization and robustness, particularly in limited training processes with limited data like our experiment. Different values for the degree of interpolation were evaluated.
Secondly, the data was standardized with a mean of zero and standard deviation of one, since previous research has shown that this can lead to improved generalization and might facilitate regularization (Vinay 2021).
Thirdly, oversampling was implemented by showing some samples more often in the training process. This approach was motivated by the observation that the distribution of the selected pixels from the pixel pipeline was non-Gaussian.
All three techniques improved convergence speed and training performance. However, they did not improve the validation performance of the trained models.

Neural Network Optimization

For each of the three main approaches, a range of different methods for improving the models used were investigated.

Firstly, multiple different model architecture were evaluated and the best performing ones were identified.

- Convolutional neural network (CNN)
- Transformer
- Simple neural network
- Deeper neural network

The CNN showed optimal performance for the whole image pipeline. For the pixel pipeline, a simpler neural network demonstrated signs of learning only for a limited number of pixels. However, for more than 50 selected pixels, a deeper neural network was required to show signs of learning. For the PCA pipeline, both the CNN and simple and deeper neural networks showed similar performances. The small Transformer-based architecture did not show any promising results for any of the pipelines.
Secondly, L2 regularization (Cortes, Mohri, and Rostamizadeh 2012) was used in when optimizing the network.
Thirdly, dropout (Srivastava et al. 2014) was introduced to multiple layers in the neural networks. Dropout is a method where certain layers are temporarily removed with a certain probability during training. This prevents overfitting.
Fourthly, batch normalization (Ioffe and Szegedy 2015) was introduced to the layers of the neural networks. Batch normalization standardizes the inputs of each layer, which has been shown to increase training performances.
Fifthly, Xavier initialization (Glorot and Bengio 2010), an initialization technique where initial weights are set to allow for effective forward and backpropagation.
While regularization, dropout, batch normalization, and Xavier initialization enhanced convergence speed and training performance, they only marginally improved validation performance and failed to consistently reach positive R^2 values.
For each architecture different loss functions, learning rates, batch sizes were evaluated.

2.4.6 Finding the Most Exciting Stimuli

Based on Walker et al. 2019 approach, optimizing towards the most exciting stimuli contained the following steps:

1. Start with a Gaussian white noise image.
2. Optimize with gradient descent towards the maximum output of the trained neural network.
3. Repeat the process with different initializations and average out the results.

Since even the best models overfitted and were not able to capture any variance in the test data, the last part of the pipeline did not show any meaningful results.

Simulated Data

3.1 Motivation

The results obtained using the dataset provided by Nathaniel Powell in Gordon Smith's lab at the University of Minnesota were disappointing. A possible reason for this could be the limited amount of data and its insufficient quality.
To evaluate the viability of the approach independently of data quality, we conducted a second experiment using artificially generated data (Antolík et al. 2024).

3.2 Experimental Setup

The experiments were based on the same pipeline as before. First: Each CNN was trained on the stimuli and a set of responses from a group of neurons. After training, an initially random input was optimized to maximize the neural activation of the model. For the results shown in Figure 3.4 the optimization was aimed at maximizing the average output of all neurons. Later, optimizing the model's output just for the first, second or third principal component of neural responses was also tested (see Figure 3.5).
The results are based on a subset of data by Antolík et al. 2024, consisting of 7,719 natural stimuli. Each of the four models was trained and evaluated separately on this dataset to produce optimized input images corresponding to the different neural response profiles.

3.3 Data

The dataset consisted of grayscale natural scene images with a resolution of 220×220 pixels. The corresponding neural responses were artificially generated for different populations of neurons in the primary visual cortex (V1). These responses were split into four categories based on both the type of neuron (excitatory or inhibitory) and cortical layer (layer 4 or layer 23). Specifically, the dataset includes responses from: excitatory neurons in layer 4 (37,500 neurons) excitatory neurons in layer 23 (37,500 neurons) inhibitory neurons in layer 4 (9,375 neurons) inhibitory neurons in layer 23

(9,375 neurons). Below sample images are shown, the full dataset can be found here Data.

Figure 3.1: Three example natural images from the simulated dataset

3.4 Model Architecture

For each of the four neuron populations, a separate CNN to predict the neural responses from the visual stimuli was trained. Each model shares the same core architecture based on the previous experiment with small adjustments, consisting of: 4 convolutional layers 2 max-pooling layers 2 fully connected layers

3.5 Results

3.5.1 Training and Validation

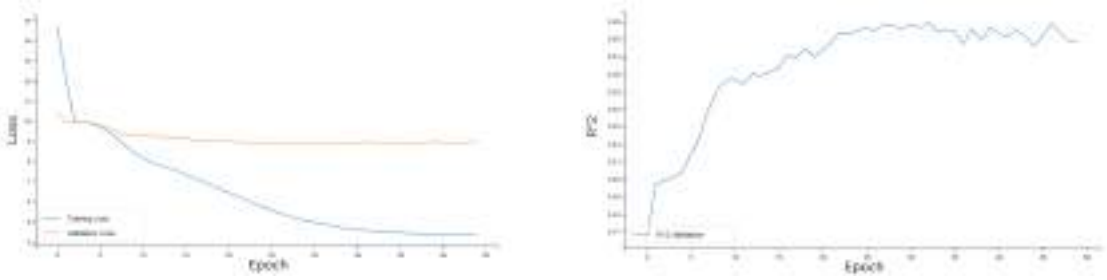


Figure 3.2: Training and validation metrics of the model trained on Exciting neurons in Layer 23

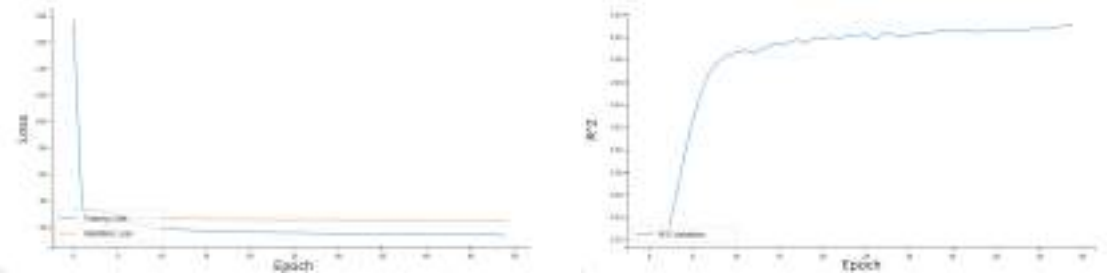
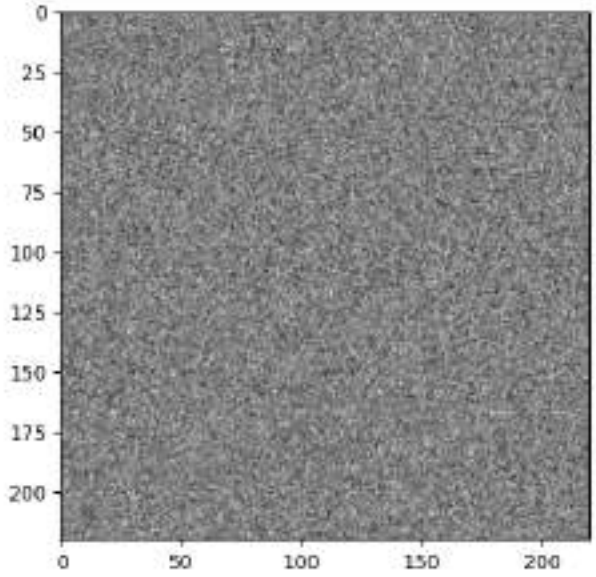


Figure 3.3: Training and validation metrics of the model trained on inhibiting neurons in layer 23

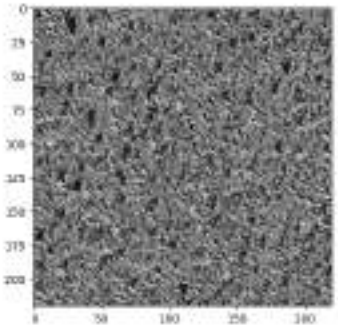
Both models achieved moderate generalization performance, as indicated by positive validation R^2 values (exciting l23: 0.2781, inhibiting l23: 0.4102) with relatively low differences between training and validation metrics, suggesting limited overfitting. This supports the hypothesis that the poor generalization observed in earlier experiments was primarily due to limitation in quality and quantity of the biological dataset.

3.5.2 Optimized Stimuli

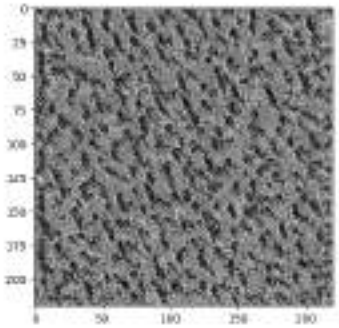
Figure 3.4 shows (a) the initial Gaussian noise image used as the starting point for optimization, and (b), (c), (d), (e) the resulting images after optimizing the input to maximize the average output of the CNN trained on each specific neuron population (excitatory layer 4, excitatory layer 23, inhibitory layer 4, inhibitory layer 23).
Compared to the initial noise image, the optimized stimuli exhibit structured patterns and textures, revealing visual features that the models have learned to be most effective in driving neural responses. These patterns resemble those observed in previous studies such as Bashivan, Kar, and DiCarlo (2019).



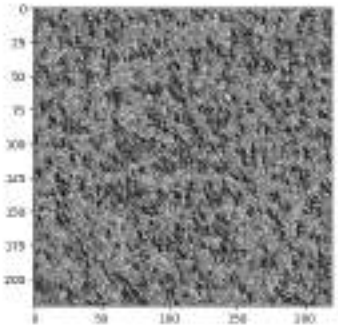
(a) Initial image



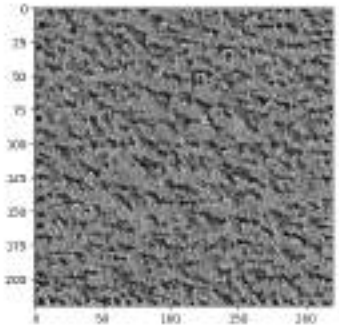
(b) Excitatory L4



(c) Excitatory L23



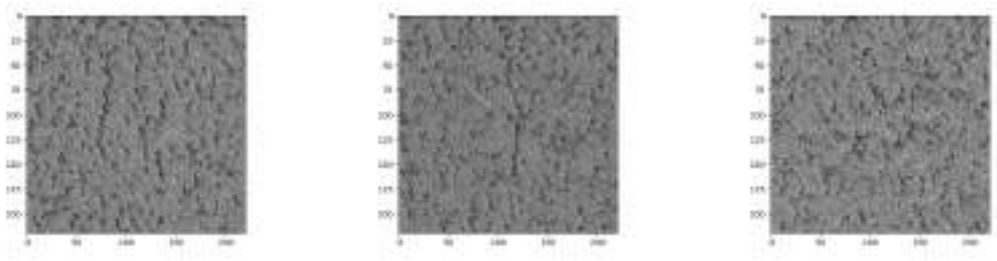
(d) Inhibitory L4



(e) Inhibitory L23

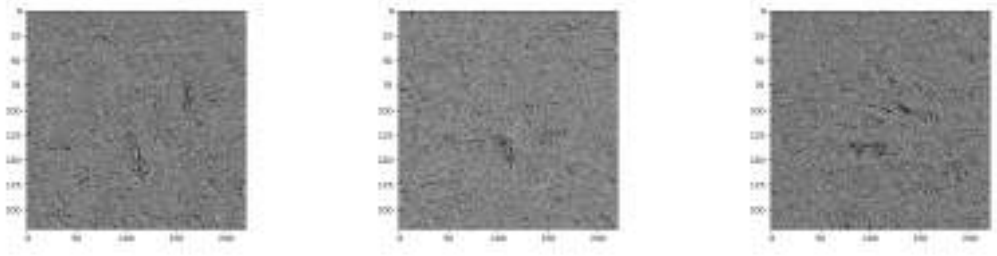
Figure 3.4: Initial input image (top) and optimized images for different neuron populations across cortical layers.

Figures 3.5 and 3.6 show the optimized images corresponding to the first three principal components (PC1, PC2, and PC3) of the neural responses for excitatory and inhibitory neurons in layer 23. In this setup, PCA was applied to the neural response data, and the same trained CNN models were used to optimize the initially random input images towards maximizing the model output along the first, second, and third principal components of the neural responses.
Compared to the population-averaged optimization in Figure 3.4, the PC-optimized images exhibit a smaller amount and more subtle structures.



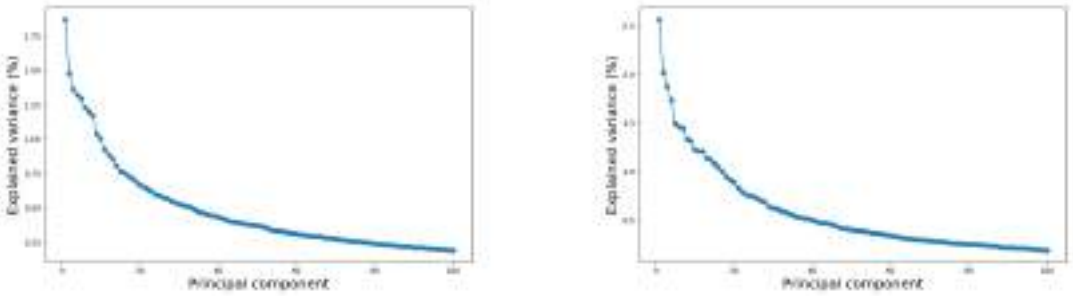
(a) PC1 optimized image (b) PC2 optimized image (c) PC3 optimized image

Figure 3.5: Optimized images corresponding to the first three principal components of excitatory layer 23 neurons.



(a) PC1 optimized image (b) PC2 optimized image (c) PC3 optimized image

Figure 3.6: Optimized images corresponding to the first three principal components of inhibitory layer 23 neurons.



(a) Excitatory L23 (b) Inhibitory L23

Figure 3.7: Explained variance per principal component for the neural responses from excitatory neurons in layer 23 and inhibitory neurons in layer 23

Discussion

In conclusion, this project established a comprehensive pipeline for modeling neural responses to visual stimuli using deep learning. A variety of architectures and regularization strategies were implemented with the goal of improving generalization performance.

Despite these efforts, the models trained in the first part of the project consistently showed substantial overfitting. Even simpler regression models failed to generalize reliably, suggesting that model complexity was not the primary limiting factor. A plausible explanation for these results is the limited size and quality of the neural data used in the first part of the project. To test this hypothesis, a follow-up experiment using simulated data was conducted.

Here, models trained on simulated data achieved moderate generalization performance. This confirms that the core pipeline is capable of learning stimulus-response mappings when trained on high-quality and sufficiently large datasets. Furthermore, the optimized stimuli generated from these models revealed structured visual patterns similar to previous findings from Bashivan, Kar, and DiCarlo (2019).

This project lays the groundwork for future experiments. The developed pipeline can be reused or extended for new datasets. In particular, applying the same methodology to a larger set of naturally recorded neural data would be promising. Another viable path for future work is to investigate whether models pretrained on neural data can be fine-tuned on smaller datasets without exhibiting substantial overfitting.

Appendix

5.1 Pipeline

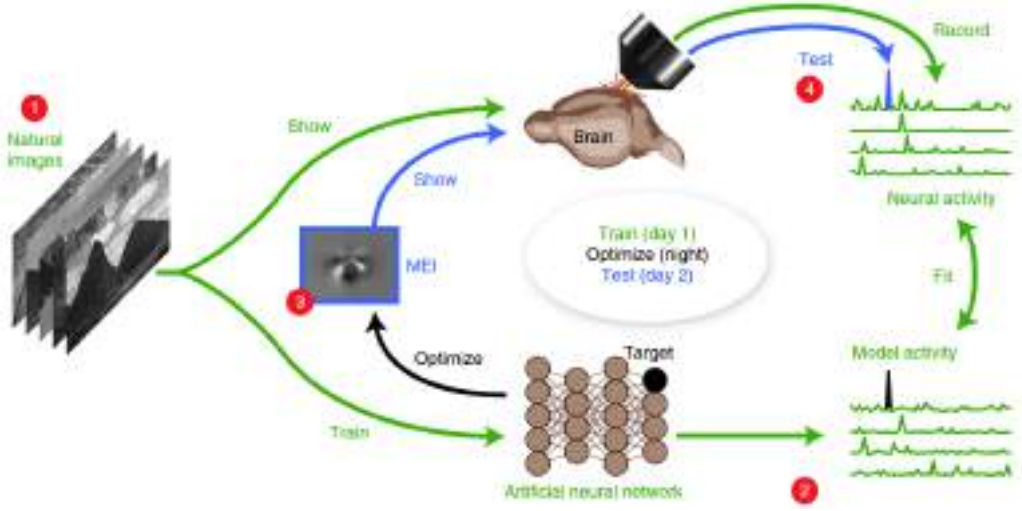


Figure 5.1: Schematic of the inception loop from Walker et al. (2019)

Bibliography

Antolík, Ján et al. (2024). "A comprehensive data-driven model of cat primary visual cortex". In: *PLOS Computational Biology* 20.8. DOI: 10.1371/journal.pcbi.1012342.

Bashivan, Pouya, Kohitij Kar, and James J. DiCarlo (2019). "Neural population control via deep image synthesis". In: *Science* 364.6439. DOI: 10.1126/science.aav9436.

Cortes, Corinna, Mehryar Mohri, and Afshin Rostamizadeh (2012). *L2 Regularization for Learning Kernels*. arXiv: 1205.2653 [cs.LG].

Glorot, Xavier and Yoshua Bengio (13–15 May 2010). "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Yee Whye Teh and Mike Titterington. Vol. 9. Proceedings of Machine Learning Research. Chia Laguna Resort, Sardinia, Italy: PMLR, pp. 249–256.

Ioffe, Sergey and Christian Szegedy (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *CoRR*. arXiv: 1502.03167.

Olmos, Andres and Frederick A. A. Kingdom (2004). "A biologically inspired algorithm for the recovery of shading and reflectance images". In: *Perception* 33, pp. 1463–1473.

Smith, G. B. et al. (2018). "Distributed network interactions and their emergence in developing neocortex". In: *Nature Neuroscience* 21.11, pp. 1600–1608. DOI: 10.1038/s41593-018-0247-5.

Srivastava, Nitish et al. (2014). "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". In: *Journal of Machine Learning Research* 15.56, pp. 1929–1958.

Stone, M. (Dec. 2018). "Cross-Validatory Choice and Assessment of Statistical Predictions". In: *Journal of the Royal Statistical Society: Series B (Methodological)* 36.2, pp. 111–133. ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1974.tb00994.x.

Vinay, Sachin (Mar. 2021). “Standardization in Machine Learning”. In.

Walker, Edgar Y. et al. (Dec. 1, 2019). “Inception loops discover what excites neurons most using deep predictive models”. In: *Nature Neuroscience* 22.12, pp. 2060–2065. ISSN: 1546-1726. DOI: 10.1038/s41593-019-0517-x.

Zhang, Hongyi et al. (2018). *mixup: Beyond Empirical Risk Minimization*. arXiv: 1710.09412 [cs.LG].